Informatica® Application Integration
July 2026

AMQP Connector Guide

Informatica Application Integration AMQP Connector Guide
July 2026

© Copyright Informatica LLC 1993, 2026

Publication Date: 2026-06-25

Table of Contents

Preface 4

Chapter 1: Introduction to AMQP Connector 5

AMQP Connector Overview 5

AMQP Implementation 5

Configuring Connectivity with Azure Service Bus 6

Create AMQP connection for Azure Service Bus in Application Integration 6

Chapter 2: AMQP Connections 11

AMQP Connections Overview 11

Basic Connection Properties 12

AMQP Connection Properties 12

Basic Event Source Properties 13

AMQP Event Source Properties 14

Basic Event Target Properties 15

AMQP Event Target Properties 15

Message Payload 15

Creating a Process to Handle AMQP Messages 16

Index 17

Preface

Read the *AMQP Connector Guide* to learn how to set up and use AMQP connections.

This guide assumes that you have an understanding of message queue systems such as ActiveMQ, and how to create connections and processes in Application Integration.

CHAPTER 1

Introduction to AMQP Connector

This chapter includes the following topics:

- AMQP Connector Overview, 5
- AMQP Implementation, 5
- Configuring Connectivity with Azure Service Bus , 6

AMQP Connector Overview

Advanced Messaging Queuing Protocol (AMQP) is a message-oriented standard with queuing, routing, reliability and security features. AMQP is a platform-agnostic protocol that allows you to facilitate business transactions by passing real-time data streams. Unlike JMS, which defines an API and a set of behaviors that a messaging implementation must provide, AMQP is a wire-level protocol.

Using the AMQP Connector, you can perform on-premises integration with AMQP brokers such as Apache ActiveMQ 5.11 or cloud integration with the Microsoft Azure service bus that implements AMQP, monitoring a message queue and handling publish/subscribe patterns for brokered messaging.

The AMQP Connector for Application Integration is based on an extensible listener framework that you can use to interact with external file systems.

You can integrate with external file systems by using the following methods:

- Listen for files at a user determined location and process the contents.
- Utilize Salesforce streaming capability to check for changes in the database environment.
- Select from a table and look for changes in a database to trigger a process.

Note: If you use ActiveMQ as your provider, you must use ActiveMQ 5.11 or later. To use RabbitMQ, you must enable the AMQP 1.0 plug-in for RabbitMQ. The latest patch release of the AMQP 1.0 plug-in from Pivotal contains the fixes for defects in the plug-in implementation. For more information, see https://github.com/rabbitmq/rabbitmq-amqp1.0/issues/31#issuecomment-234960297 or contact Pivotal.

AMQP Implementation

AMQP Connector supports AMQP by using a Camel event listener. Processes that connect to AMQP can triggered upon an event such as arrival of an AMQP message or a file in a file system.

AMQP enables one connection or one broker to support multiple queues using an over-the-wire format, similar to JMS providers, ApacheMQ or a Microsoft Azure service bus. You can access different file systems and enable different types of file processing, based on the connector definitions. These connectors run on an agent and where applicable, are accessible from the cloud.

Note: When you connect to files, your performance might be impacted by the speed of the underlying file systems.

When you publish a process that uses the AMQP connector, the binding details are stored in a catalog file (`entrypoints.xml`) that determines how to route the message.

The message payload is defined by the connector.

You can use the same AMQP connection to connect to the source as well as a target. When you trigger a process, "fan out" is supported, so one message can spawn multiple processes.

Configuring Connectivity with Azure Service Bus

You can configure Process Server connectivity with Azure Service Bus by using an AMQP connection. Microsoft Azure supports AMQP version 1.0 for its Service Bus implementation. AMQP connector uses both Service Bus queues and topics for configuration, but with some limitations.

Create AMQP connection for Azure Service Bus in Application Integration

You can connect to Azure Service Bus queue or topic using the AMQP connector in Application Integration. After you have created the Azure Service Bus connector, you can configure Process Server connectivity with Azure Service Bus by using an AMQP connection.

Ensure that the following settings are performed when you create the AMQP connection:

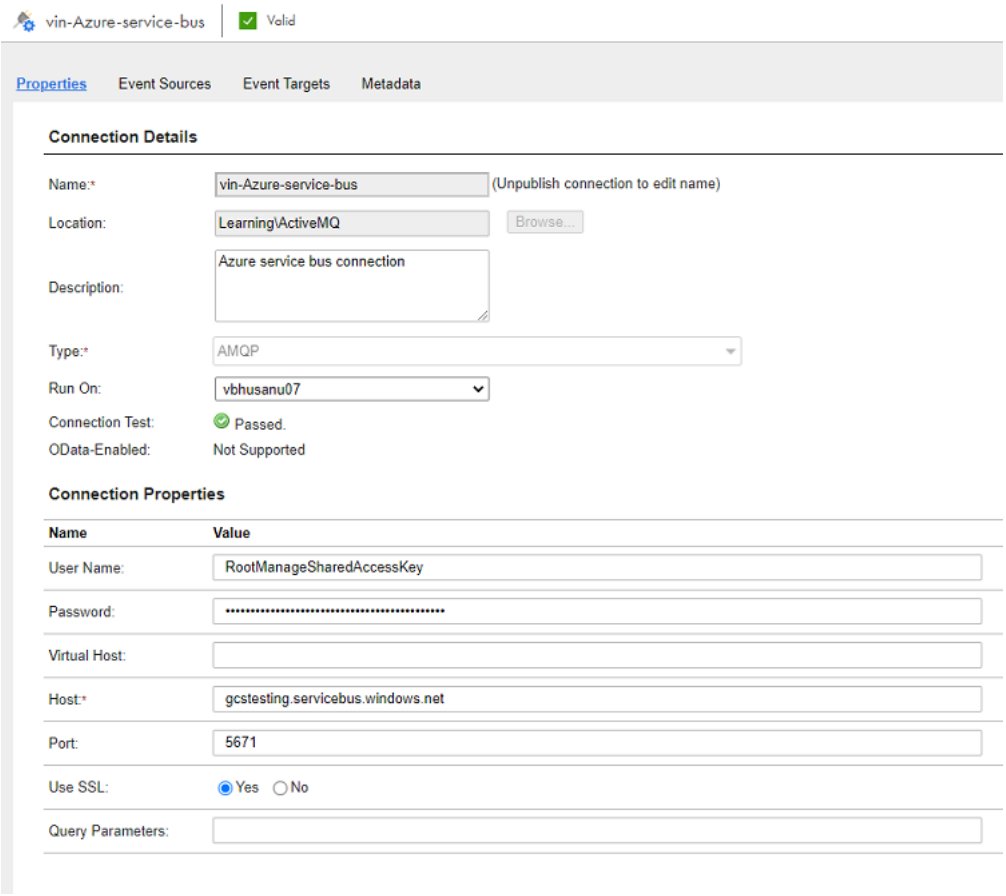
On Azure Portal

- Ensure that the Azure Service Bus has a queue created with the name **informatica-test-queue** for the test connection to be successful.
- Disable the **Enable sessions** option when you create a Azure Service Bus topic in Azure portal.

In Azure Service Bus Connection Properties

1. Enter the Shared Access Policy name as the username.
2. Enter the primary key as the password.
3. Enter the value in the **Host** field in the `<service bus name>.servicebus.windows.net` format.
4. Enter the value in the Port field as `5671` .
5. Set the **Use SSL** field to **Yes**.

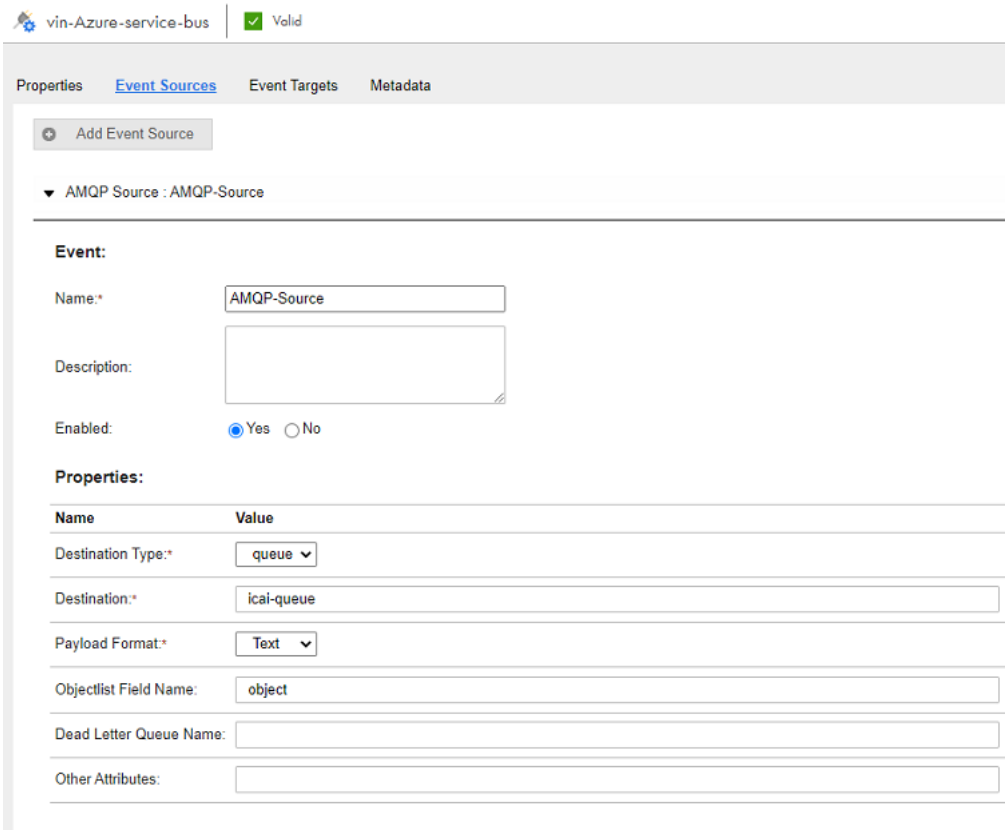
The following image the sample values that you must enter when you configure the following values in the Azure Service Bus Connection source properties:



Configuring the Azure Bus Service queue

When you configure the Azure Bus Service queue, In the **Destination Type** field, select `queue`.

The following image the sample values that you must enter when you configure the the Azure Bus Service queue:

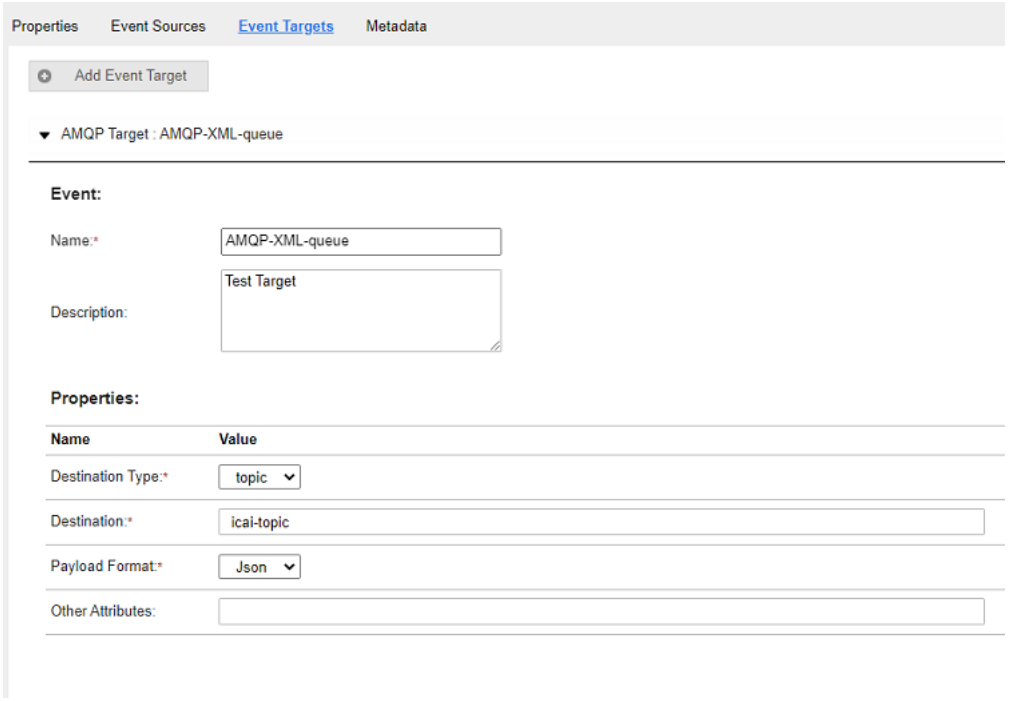


Configuring the Azure Bus Service topics

When you configure the Azure Bus Service topic, configure the following values in the event target and event source properties:

1. To connect to a messaging topic, you must first create a subscription to that topic. A subscription allows you to receive messages published to the topic. In order to avoid errors in the connection, don't enable sessions for it when you create subscriptions.

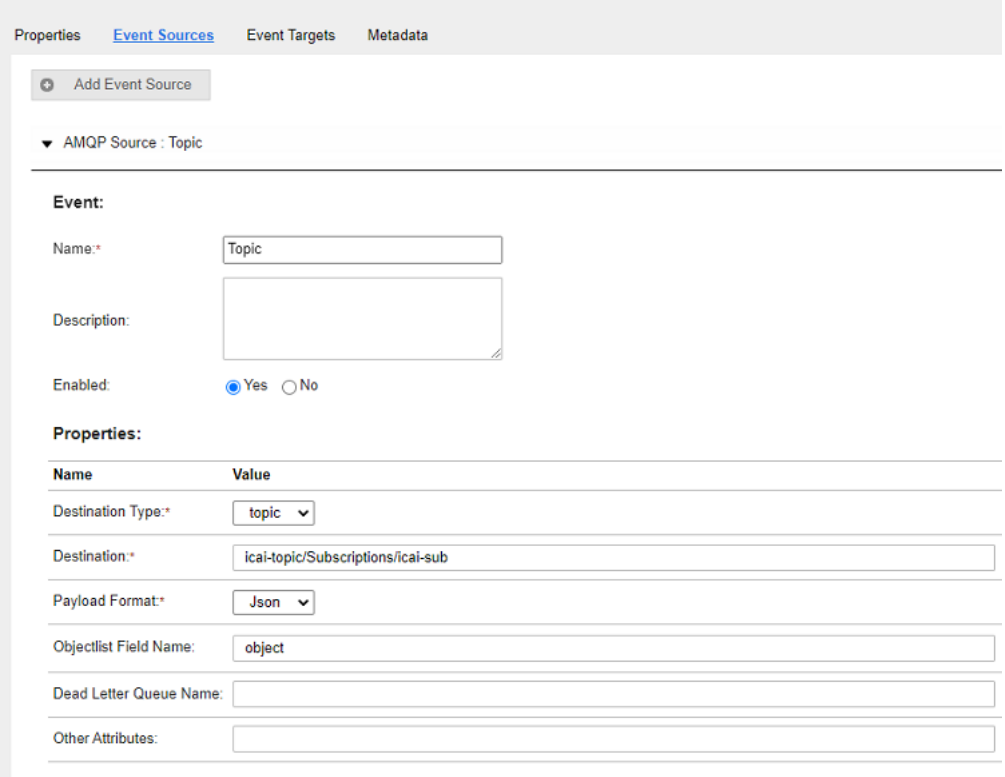
2. When you configure the AMQP connection for an event target that acts as a Producer, you must select the **Destination Type** field, select `topic` as shown in the following image:



Note: Additional details such as the queue name or subscription are not required.

3. When you configure the AMQP connection for an event source that acts as a Consumer, you must select the **Destination Type** field, select `topic`. However, you must enter the value in the Destination type in the following format: `<Topic name >/Subscriptions/<Subscription name>`

The following image the sample values that you must enter when you configure the AMQP connection for an event source that acts as a Consumer:



In the above image the topic name is `icai-topic` and the subscription name is `icai-sub`.

CHAPTER 2

AMQP Connections

This chapter includes the following topics:

- AMQP Connections Overview, 11
- Basic Connection Properties, 12
- AMQP Connection Properties, 12
- Basic Event Source Properties, 13
- AMQP Event Source Properties, 14
- Basic Event Target Properties, 15
- AMQP Event Target Properties, 15
- Message Payload, 15
- Creating a Process to Handle AMQP Messages, 16

AMQP Connections Overview

You can use an AMQP connection to connect to an AMQP stream and send messages to a queue or delete messages in a queue.

After you create an AMQP connection, you can validate and save the connection.

You can then publish the AMQP connection and click the **Metadata** tab to view the generated process objects for the connection.

Basic Connection Properties

The following table describes the basic properties that you can configure on the **Properties** tab of the connection creation page:

Properties	Description
Name	Required. Unique name for the AMQP connection that identifies it in the Process Designer. The name must start with an alphabet and can contain only alphabets, numbers, or hyphens (-).
Location	Optional. The location of the project or folder where you want to save the connection. Click **Browse** to select a location. If the **Explore** page is currently active and a project or folder is selected, the default location for the connection is the selected project or folder. Otherwise, the default location is the location of the most recently saved asset.
Description	Optional. Description of the connection.
Type	Required. The type of connection that you want to use for the connector or service connector. Select **AMQP**.
Runtime Environment	Required. The runtime environment for the connection. You can run the connection on a Secure Agent group, or a Secure Agent machine.
Connection Test	Not supported for AMQP Connector.
OData-Enabled	Not supported for AMQP Connector.

After you configure the basic properties, you must also define the following properties:

- The properties applicable to the AMQP connection type
- The event source properties and event target properties for the AMQP connection

After you publish the AMQP connection, the **Metadata** tab displays the generated process objects.

AMQP Connection Properties

The following table defines the AMQP connection properties that you must configure in the **Connection Properties** section:

Property	Description
User Name	Optional. The user name from the SASL exchange that occurs in the AMQP methods `connection.secure` and `connection.secure-ok`.
Password	The password from the SASL exchange that occurs in the AMQP methods `connection.secure` and `connection.secure-ok`.
Virtual Host	The `virtual host` (or vhost) that specifies the namespace for the entities (exchanges and queues) and partitions message broker data into distinct sets to which the AMQP protocol refers.

Property	Description
Host	Required. The host for the underlying TCP connection. For a connection that uses a single host, enter the host name in the **Host** field and the port number in the **Port** field. To enable high availability, enter a comma-separated list of host:port pairs to fail over to the next available host automatically when the first host is unavailable. For example, use the following format: `<host1>:<port1>,<host2>:<port2>,<host3>:<port3>`
Port	The port number used to communicate with the AMQP server. This field is applicable only for a connection that uses a single host. Default is **5672**.
Use SSL	Determines whether to use the amqps:// protocol for the connection. Default is **No**.
Query Parameters	Specify other parameters that define how to connect to servers, if required by the AMQP client. For example, the standard URI query syntax might be used to provide additional information for the client to successfully connect.

Basic Event Source Properties

You can add one or more event sources for a connection. An event source serves as a start event that listens or monitors a specified location for new files or messages. After you define an event source for a connection, you can publish the connection only on a Secure Agent and not on the Cloud Server. You can then access the event source in a process and deploy the process only on a Secure Agent to consume the process objects generated by the event source downstream.

To create event sources for a connection, from the **Event Sources** tab, click **Add Event Source**, and select the event source type from the available list.

The following table describes the event source properties that are available for all event source types:

Property	Description
Name	Required. Unique name for the event source.
Description	Optional. Description for the event source.
Enabled	Select **Yes** to make the event source available immediately after it is published. Select **No** to disable the event source until you are ready to use it. Default is **Yes**.

You can view the status of each event source in the published connection. If the status of the event source is stopped, you can republish the connection and restart the event source. When you republish the connection, all the event sources in the connection start by default.

For more information about starting and stopping event sources in a listener-based connection, see *Connectors for Cloud Application Integration and Monitor*.

AMQP Event Source Properties

For each AMQP event source that you create, you set the following properties to dequeue messages or subscribe to the service. Each event source defined is available to use in a process that handles the messages.

Event Source Property	Description
Enable Load Balancing	Required. Determines whether the connection must be deployed on all the Secure Agents in a group or on the selected Secure Agent for load balancing. You must enable this option only if the Process Server uses a Secure Agent Cluster configuration. Default is **No**. After you publish a connection and run a process, if you toggle the load balancing option, you might see duplicate messages. To avoid this issue, Informatica recommends that you create a new connection if you want to change the load balancing option.
Custom Message ID	Optional. Specify the custom message ID to override the default JMS message ID. Ensure that the message header contains the custom message ID.
Destination Type	Required. Indicates the AMQP destination type. Select one the following values: - queue - topic
Destination	Required. The name of the queue or topic that you have defined in AMQP. For example, if the destination type is a topic in ActiveMQ, you would specify the Destination value in the following format: `topic://<topic-name>`
Payload Format	Required. Specifies the type of message payload format that you expect on the destination. Select one of the following values: - XML - JSON - Text - Binary
Objectlist Field Name	Optional. If the payload format is JSON, the queue or topic might accept a JSON message that contains an array of objects. If it does, enter the value to be used as the field name of the object list in the AMQP message body.
Dead Letter Queue Name	Optional. The name of a queue that you have defined in AMQP where you can route messages that encounter a failure during processing. The question mark `(?)` character is prohibited in this field.
Other Attributes	Optional. The other attributes that you want to use for the event source. Specify the attributes in key-value pairs. Separate multiple attributes with an ampersand character `(&)`. For example, enter the following phrase to specify the number of concurrent consumers and the maximum number of concurrent consumers: `concurrentConsumers=1&maxConcurrentConsumers=2` For more information about the attributes that you can specify, see the Apache Camel JMS documentation. Contact Informatica Global Customer Support if you need assistance.

Basic Event Target Properties

For each connection that you define, you can include one or more event targets that specify operations for writing files or messages, or when the event target is called from a process. For example, you might define an event target that reads from a process object and writes to comma-delimited files.

To set event target properties for the connection, from the **Event Targets** tab, click **Add Event Target**, and select the event target type from the available list.

The following table describes the basic properties:

Property	Description
Name	Required. Unique name for the event target.
Description	Optional. Description of the event target.

AMQP Event Target Properties

The following table describes the AMQP event target properties:

Property	Description.
Destination Type	Required. Indicates the AMQP destination type. Select queue or topic.
Destination	Required. The name of the queue or topic that you have defined in AMQP. For example, if the destination type is a topic in ActiveMQ, you would specify the destination value in the following format: `topic://<topic-name>`
Payload Format	Required. Specifies the type of message payload format that you expect on the destination. Select one the following values: - XML - JSON - Text - Binary
Other Attributes	Optional. If you need to set other attributes for the event target, you can supply them here. Contact Informatica Global Customer Support for assistance.

Message Payload

The event received by the process is an object with a payload that represents the message data (the AMQP message).

Similar to other connectors, the message payload for AMQP might be made accessible to the process as a process object of type $any.

For example:

```
[
{"firstName":"cindy", "lastName": "lou", "department": "qa", "id":1},
{"firstName":"billy", "lastName": "joe", "department": "dev", "id":12}
]
```

or in XML, you can write it as follows:

```
<list>
 <employee>
   <firstName>cindy</firstName>
   <lastName>lou</lastName>
   <department>qa</department>
   <id>1</id>
 </employee>
 <employee>
   <firstName>billy</firstName>
   <lastName>joe</lastName>
   <department>dev</department>
   <id>12</id>
 </employee>
</list>
```

If the payload is XML, the field name is the element name of the list, for example, 'employee'.

If the payload is JSON, the array does not have a field name as defined by the payload and you can specify an Object Field Name for the Event Source so that the implicit object field name can be set.

Creating a Process to Handle AMQP Messages

To create a process that handles the AMQP messages, follow these steps:

1. Create a process and select a Secure Agent where the AMQP broker should run and other properties of the process.
2. On the Start tab, choose to **Start Via** Event as AMQP consumes events.
   You can then see the available event sources (grouped by *Connection:event name*).

Process Example

To queue messages on the AMQP queue, you might define a process that includes:

- An Assignment step to assign the message to the orders queue.
- A Wait step to include a timer.

The event target can be handled like other service calls. The input argument is a process object that defines the data required for the transport.

INDEX

A

AMQP
  Basic Connection properties 12

B

Basic connection properties
  AMQP 12